Templates
Ch 14

Object Oriented Programming

CSE-4301

Presented by

Rafsanjany Kushol

Assistant Professor

Templates

- Templates make it possible to use one function or class to handle many different data types.
- Rewriting the same function body over and over for different types is time-consuming and wastes space.
- If you find you've made an error in one such function, you'll need to remember to correct it in each function body.
- It would be nice if there were a way to write such a function just once, and have it work for many different data types. This is exactly what function templates do for you.

Function Templates

Suppose you want to write a function that returns the absolute value of two numbers.

```
int abs(int n)                  //absolute value of ints
   {
   return (n<0) ? -n : n;   //if n is negative, return -n
   }

long abs(long n)               //absolute value of longs
   {
   return (n<0) ? -n : n;
   }

float abs(float n)             //absolute value of floats
   {
   return (n<0) ? -n : n;
   }
```

. . .

Function Templates

```
template <class T>
T abs(T n)
{
    return (n < 0) ? -n : n;
}
int int1 = 5;
int int2 = -6;
long lon1 = 70000L;
long lon2 = -80000L;
double dub1 = 9.95;
double dub2 = -10.15;
//calls instantiate functions
cout << "\nabs(" << int1 << ")=" << abs(int1);   //abs(int)
cout << "\nabs(" << int2 << ")=" << abs(int2);   //abs(int)
cout << "\nabs(" << lon1 << ")=" << abs(lon1);   //abs(long)
cout << "\nabs(" << lon2 << ")=" << abs(lon2);   //abs(long)
cout << "\nabs(" << dub1 << ")=" << abs(dub1);   //abs(double)
cout << "\nabs(" << dub2 << ")=" << abs(dub2);   //abs(double)
```

Here's the output of the program:

```
abs(5)=5
abs(-6)=6
abs(70000)=70000
abs(-80000)=80000
abs(9.95)=9.95
abs(-10.15)=10.15
```

What the Compiler Does

- Nothing right away. The function template itself doesn't cause the compiler to generate any code.
- It simply remembers the template for possible future use.
- When the compiler sees such a function call, it knows that the type to use is *int*.
- So it generates a specific version of the function for type int, substituting int wherever it sees the name T in the function template.
- This is called ***instantiating*** the function template, and each instantiated version of the function is called a ***template function***.

Function Templates with Multiple Arguments

```
template <class atype>
int find(atype* array, atype value, int size)
{
    for(int j=0; j<size; j++)
        if(array[j]==value)
            return j;
    return -1;
}
```

Function Templates with Multiple Arguments

```
char chrArr[] =    {1, 3, 5, 9, 11, 13};  //array
char ch = 5;                              //value to find
int intArr[] =     {1, 3, 5, 9, 11, 13};
int in = 6;
long lonArr[] =    {1L, 3L, 5L, 9L, 11L, 13L};
long lo = 11L;
double dubArr[] = {1.0, 3.0, 5.0, 9.0, 11.0, 13.0};
double db = 4.0;
cout << "\n  5 in chrArray: index=" << find(chrArr, ch, 6);
cout << "\n  6 in intArray: index=" << find(intArr, in, 6);
cout << "\n 11 in lonArray: index=" << find(lonArr, lo, 6);
cout << "\n  4 in dubArray: index=" << find(dubArr, db, 6);
```

```
 5 in chrArray: index=2
 6 in intArray: index=-1
11 in lonArray: index=4
 4 in dubArray: index=-1
```

Template Arguments Must Match

```
int intarray[] = {1, 3, 5, 7};          //int array
float f1 = 5.0;                         //float value
int value = find(intarray, f1, 4);      //uh, oh
```

```
find(int*, int, int);
```

```
find(int*, float, int);
```

More Than One Template Argument

```
template <class atype, class btype>
btype find(atype* array, atype value, btype size)
   {
   for(btype j=0; j<size; j++)    //note use of btype
      if(array[j]==value)
         return j;
   return static_cast<btype>(-1);
   }
```

Why Not Macros?

```
#define abs(n) ( (n<0) ? (-n) : (n) )
```

- Macros don’t perform any type checking.
- Also, the type of the value returned isn’t specified, so the compiler can’t tell if you’re assigning it to an incompatible variable

Class Templates

```
class Stack
   {
   private:
      int st[MAX];          //array of ints
      int top;              //index number of top of stack
   public:
      Stack();              //constructor
      void push(int var);   //takes int as argument
      int pop();            //returns int value
   };

class LongStack
   {
   private:
      long st[MAX];         //array of longs
      int top;              //index number of top of stack
   public:
      LongStack();          //constructor
      void push(long var);  //takes long as argument
      long pop();           //returns long value
   };
```

Class Templates

```
template <class Type>
class Stack
   {
   private:
      Type st[MAX];              //stack: array of any type
      int top;                   //number of top of stack
   public:
      Stack()                    //constructor
         { top = -1; }
      void push(Type var)        //put number on stack
         { st[++top] = var; }
      Type pop()                 //take number off stack
         { return st[top--]; }
   };
```

Class Templates

```
Stack<float> s1;       //s1 is object of class Stack<float>

s1.push(1111.1F);      //push 3 floats, pop 3 floats
s1.push(2222.2F);
s1.push(3333.3F);
cout << "1: " << s1.pop() << endl;
cout << "2: " << s1.pop() << endl;
cout << "3: " << s1.pop() << endl;

Stack<long> s2;        //s2 is object of class Stack<long>

s2.push(123123123L);   //push 3 longs, pop 3 longs
s2.push(234234234L);
s2.push(345345345L);
cout << "1: " << s2.pop() << endl;
cout << "2: " << s2.pop() << endl;
cout << "3: " << s2.pop() << endl;
```

Class Name Depends on Context

```
template <class Type>
class Stack
   {
   private:
      Type st[MAX];                 //stack: array of any type
      int top;                      //number of top of stack
   public:
      Stack();                      //constructor
      void push(Type var);          //put number on stack
      Type pop();                   //take number off stack
   };
```

Class Name Depends on Context

```
template<class Type>
Stack<Type>::Stack()              //constructor
   {
   top = -1;
   }
//--------------------------------------------------------------
template<class Type>
void Stack<Type>::push(Type var) //put number on stack
   {
   st[++top] = var;
   }
//--------------------------------------------------------------
template<class Type>
Type Stack<Type>::pop()           //take number off stack
   {
   return st[top--];
   }
```

Storing User-Defined Data Types

```
template<class TYPE>                    //struct "link<TYPE>"
struct link                             //one element of list
   {
   TYPE data;                           //data item
   link* next;                          //pointer to next link
   };


template<class TYPE>                    //class "linklist<TYPE>"
class linklist                          //a list of links
   {
   private:
      link<TYPE>* first;                //pointer to first link
   public:
      linklist()                        //no-argument constructor
         { first = NULL; }              //no first link
      void additem(TYPE d);             //add data item (one link)
      void display();                   //display all links
   };
```

Storing User-Defined Data Types

```
int main()
    {                                   //lemp is object of
    linklist<employee> lemp;            //class "linklist<employee>"
    employee emptemp;                   //temporary employee storage
    char ans;                           //user's response ('y' or 'n')

    do
        {
        cin >> emptemp;                 //get employee data from user
        lemp.additem(emptemp);          //add it to linked list lemp
        cout << "\nAdd another (y/n)? ";
        cin >> ans;
        } while(ans != 'n');            //when user is done,
    lemp.display();                     //display entire linked list
    cout << endl;
```

. . .

Thank You

Any questions?

kushol@iut-dhaka.edu